

TUTORIAL FIVE**Process Synchronization- Part II**

1. Alice is playing a game that involves pairing apples with oranges. The game produces apples and oranges in separate baskets at random instants. For Alice to get points, she must **pair exactly 1 apple with 2 oranges** by picking them from the respective baskets. Each basket is protected by a semaphore; A for the apple basket and O for the orange basket. To access a basket, Alice must first acquire the corresponding semaphore.

Complete the code in the below figure to help Alice to play this game. You should only use the operations Wait(A), Wait(O), Signal(A) and Signal(O) in the boxes provided. You may use any number of these operations (including zero) in each of the boxes. **Your solution must be deadlock-free even in the presence of other players who may be executing a different code.**

```

1
// Pick 1 apple

2
If (atleast 2 oranges in basket) {
** Pick 2 oranges and pair with apple. **

3
} else {

4
** Drop 1 apple back in basket **

5
}

```

2. Consider the following implementations of the system calls Wait() and Signal().

Wait (Semaphore S)	Signal (Semaphore S)
<pre> { S.value--; If (S.value < -1) { block(); } } </pre>	<pre> { S.value++; If (S.value <= 0) { wakeup(); } } </pre>

Suppose these system calls are guaranteed to be atomic and a Semaphore S in this system is used to protect critical sections as shown below. Assume S is initialized to 0. Explain if this solution ensures mutual exclusion and progress with justifications?

```

Process Pi:

While(1)
{
    Wait(S);
    Critical-section for Pi
    Signal(S);
    Remainder-section for Pi
}

```

3. Describe how *Wait(S)* and *Signal(S)* of a semaphore can be implemented using a *TestAndSet* instruction, given the below semaphore structure definition.
Hint: *The semaphore value and the process queue L are shared variables among different processes when those processes access the semaphore.*

```
typedef struct {  
    int value;  
    struct process *L;  
} semaphore;
```